

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 12. Sets

Lecture: 132.Set Methods - Adding and Deleting

Section 1: Lecture Summary

This lecture discusses various **methods for adding and deleting elements in a Python set**. Since sets are unordered, you cannot use indexing or slicing to modify them. Instead, Python provides specific set methods to add elements, update a set with multiple elements, copy a set, and remove elements. The key methods introduced are:

- Adding methods: ``add()``, ``update()``, and ``copy()``.
- Removing methods: ``pop()``, ``discard()``, ``remove()``, and ``clear()``.

The lecture explains how each method works, their parameters, and differences, especially between the similar ``discard()`` and ``remove()`` methods regarding error behavior when elements don't exist in the set. There are also examples demonstrating handling single elements, tuples, and iterables with these methods.

Section 2: Key Concepts and Explanations

- Sets and Their Nature

- Sets are **unordered collections**; hence, they do not support indexing or slicing for element addition or deletion.

- Adding Elements to Sets

- `.add(element)`

Adds a **single element** to the set.

- Passing multiple arguments causes a `TypeError`.
- You can add a tuple as a single element (e.g., `(60, 70)`), but not a list.

- `.update(iterable)`

Adds multiple elements from an **iterable** (list, tuple, string, etc.).

- Adds elements one by one, not the iterable as a whole.
- e.g., `update((60, 70))` adds `60` and `70` separately.
- If a string is given, each character is added as an individual element.

- `.copy()`

Returns a **shallow copy** of a set.

- The new set is independent (different ID) but elements are the same.
- Changes to the new set do not affect the original.

- Removing Elements from Sets

- `.pop()`

Removes and returns an arbitrary element from the set (because sets are unordered).

- Does not take parameters.
- If the set is empty, it raises a `KeyError`.

- `.discard(element)`

Removes an element if it exists; **does not raise error** if element is missing.

- Returns `None` when element is not found, silently ignoring the removal.

- `.remove(element)`

Removes an element.

- Raises a `KeyError` if the element is **not present**.
- You must handle this exception or ensure the element exists before calling.
- Takes only one argument; multiple element removal attempts raise a `TypeError`.
- Can remove a tuple element if it exists as a single element.

- `.clear()`

Removes **all elements** from the set, resulting in an empty set.

- The set remains a set (not an empty dictionary).
- Printing an empty set shows `set()`.

Section 3: Example Code and Use Cases

Creating a set

Adding one element with add()

Trying to add multiple elements with add() (causes error)

`S1.add(60, 70)` `TypeError: add() takes exactly one argument (2 given)`

Adding a tuple as a single element with add()

Adding multiple elements with update()

Updating with a string adds each character separately

Copying a set

Removing elements

`pop()`: removes and returns arbitrary element

`discard()`: removes element if present, no error if missing

`remove()`: removes element, raises error if missing

`S1.remove(100)` `Raises KeyError: 100 (handle exception if used)`

Removing a tuple element

`clear()`: removes all elements

```
S1 = {10, 20, 30, 40, 50}
print(len(S1)) # Output: 5
```

```
S1.add(60)
print(S1)      # Output: {10, 20, 30, 40, 50, 60}
print(len(S1)) # Output: 6
```

```
S1.add((60, 70))
print(S1) # Output includes (60, 70) as one element
```

```
S1.update((80, 90))
print(S1) # Output includes 80 and 90 as separate elements
```

```
S1.update('ruby')
print(S1) # Output includes 'r', 'u', 'b', 'y' as separate elements
```

```
S2 = S1.copy()
print(id(S1), id(S2)) # Different IDs, independent sets
```

```

removed_element = S1.pop()
print(removed_element)
print(S1)

S1.discard(30) # Removes 30
S1.discard(70) # 70 not present, no error

print(S1)

S1.remove(20) # Removes 20

print(S1)

S1.remove((60, 70))
print(S1)

S1.clear()
print(S1) # Output: set()

```

This code demonstrates adding individual elements and groups to sets, copying sets, and the various methods for removing elements and their behavior.

Section 4: Key Takeaways

- Sets in Python are unordered, so item modification by index/slicing is not possible.
- Use `.add()` to insert a **single element**; `.update()` to insert **multiple elements from an iterable**.
- `.copy()` creates a shallow copy of the set (independent object).
- `.pop()` removes and returns an arbitrary element from the set without arguments.
- `.discard()` removes an element if present; no error if absent.

- `.remove()` removes an element; raises `KeyError` if the element is not found.
- Use `.clear()` to remove **all elements** from the set.
- Adding or removing multiple elements at once directly (e.g., `.add(60, 70)` or `.remove(20, 30)`) results in a `TypeError`.
- A tuple can be added or removed as a **single element** in a set.
- Always handle potential exceptions with `.remove()` when removing an element that might not exist.
- Empty sets print as `set()` (different from empty dictionaries `{}`).

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



Adding & Deleting

Set Methods

Adding & Deleting

Set Methods

Adding & Deleting

Set Methods

add(element)

Adding & Deleting

Set Methods

add(element)

update (iterable)

Adding & Deleting

Set Methods

add(element)

update (iterable)

copy()

Adding & Deleting

Set Methods

add(element)

update (iterable)

copy()

pop()



Adding & Deleting

Set Methods

- add**(element)
- update** (iterable)
- copy**()
- pop**()
- discard**(element)



Adding & Deleting

Set Methods

- add**(element)
- update** (iterable)
- copy**()
- pop**()
- discard**(element)
- remove**(element)

Adding & Deleting

Set Methods

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

adding & Deleting

Set Methods

add(element)

update (iterable)

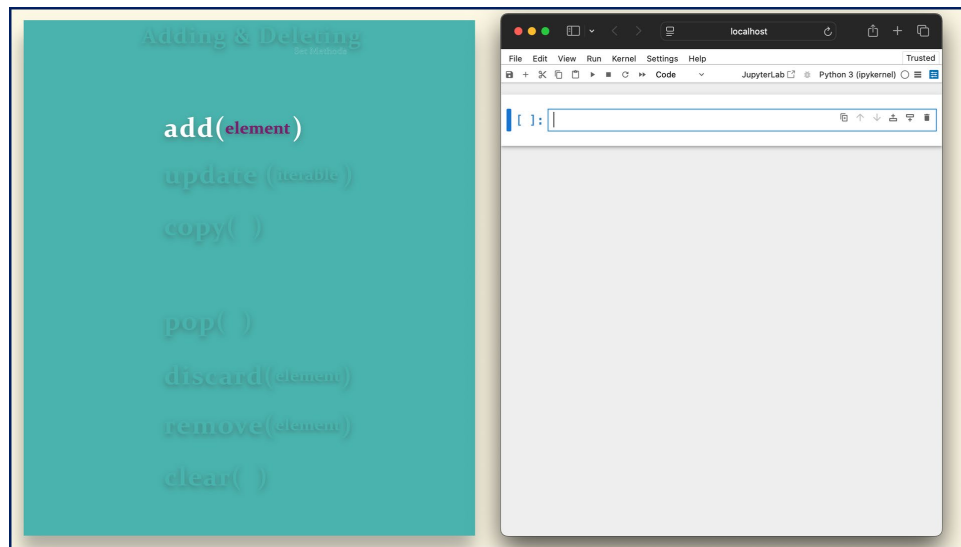
copy()

pop()

discard(element)

remove(element)

clear()



Adding & Deleting

`add(element)`
`update (locable)`
`copy()`

`pop()`
`discard(element)`
`remove(element)`
`clear()`

```
[1]: S1 = {10,20,30,40,50}
```

```
{ }:
```

S1

10	20	30	40	50
----	----	----	----	----

Adding & Deleting

`add(element)`
`update (locable)`
`copy()`

`pop()`
`discard(element)`
`remove(element)`
`clear()`

```
[1]: S1 = {10,20,30,40,50}
```

```
[2]: len(S1)
```

```
[2]: 5
```

```
{ }:
```

S1

10	20	30	40	50
----	----	----	----	----

Adding & Deleting

`add(element)`
`update (mutable)`
`copy()`

`pop()`
`discard(element)`
`remove(element)`
`clear()`

```
[1]: S1 = {10,20,30,40,50}
```

```
{ 1: |
```

S1

10	20	30	40	50
----	----	----	----	----

S1.add()

Adding & Deleting

`add(element)`
`update (mutable)`
`copy()`

`pop()`
`discard(element)`
`remove(element)`
`clear()`

```
[1]: S1 = {10,20,30,40,50}
```

```
[2]: S1.add(60)
```

```
{ 1: |
```

S1

10	20	30	40	50	60
----	----	----	----	----	----

S1.add(60)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.add(60)
[3]: S1
[3]: {10, 20, 30, 40, 50, 60}
[4]:
```

S1

10 20 30 40 50 60

S1.add(60)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.add(60)
[3]: S1
[3]: {10, 20, 30, 40, 50, 60}
[4]: len(S1)
[4]: 6
[5]:
```

S1

10 20 30 40 50 60

S1.add(60)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[]:

S1

10 20 30 40 50

S1.add()

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[]:

S1

10 20 30 40 50

S1.add(60,70)

Adding & Deleting

Set Methods

add(element)

update (Iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[3]: S1.add(60,70)

TypeError

most recent call last

Cell In[3], line 1

----> 1 S1.add(60,70)

TypeError: set.add() takes exactly one argument (2 given)

Traceback (

S1 10 20 30 40 50

S1.add(60,70)

adding & Deleting

Set Methods

add(element)

update (Iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.add((60,70))

S1 (60,70) 10 20 30 40 50

S1.add((60,70))

Adding & Deleting

add(element)

update(element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

{ }:

S1

10 20 30 40 50

S1.update((60,70))

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.update((60,70))

{ }:

S1

10 20 30 40 50 60 70

S1.update((60,70))

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.update((60,70))

[3]: S1

[3]: {10, 20, 30, 40, 50, 60, 70}

[]:

S1

10 20 30 40 50 60 70

S1.update((60,70))

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[]:

S1

10 20 30 40 50

Adding & Deleting

add(element)

update(iterable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
{ }:
```

S1 10 20 30 40 50

S1.update('ruby')

Adding & Deleting

add(element)

update(iterable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[2]: S1.update('ruby')
```

```
{ }:
```

S1 10 20 30 40 50 b r u y

S1.update('ruby')

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.update('ruby')
[3]: S1
[3]: {10, 20, 30, 40, 50, 'b', 'r', 'u', 'y'}
```

S1

10

20

30

40

50

b

r

u

y

S1.update('ruby')

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[ ]:
```

Adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[]:

S1

10 20 30 40 50

adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[]:

S1

10 20 30 40 50

S2=S1.copy()

Adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S2 = S1.copy()
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

S2

10	20	30	40	50
----	----	----	----	----

S2=S1.copy()

Adding & Deleting

add(element)

update(element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S2 = S1.copy()
[3]: id(S1)
[3]: 4568573408
```

{ 1: |

S1

1020304050

S2

1020304050

S2=S1.copy()

Adding & Deleting

add(element)

update(element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S2 = S1.copy()
[3]: id(S1)
[3]: 4568573408
[4]: id(S2)
[4]: 4568574080
```

{ 1: |

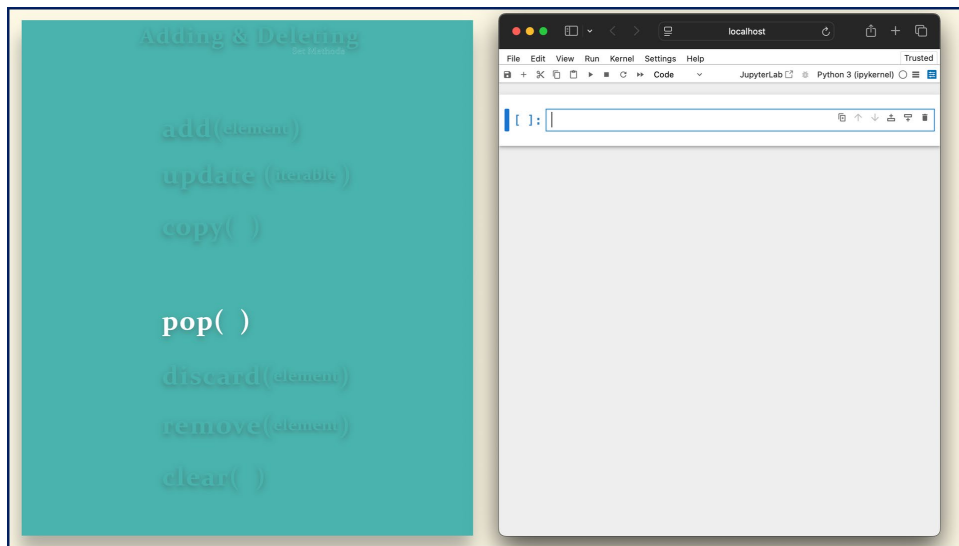
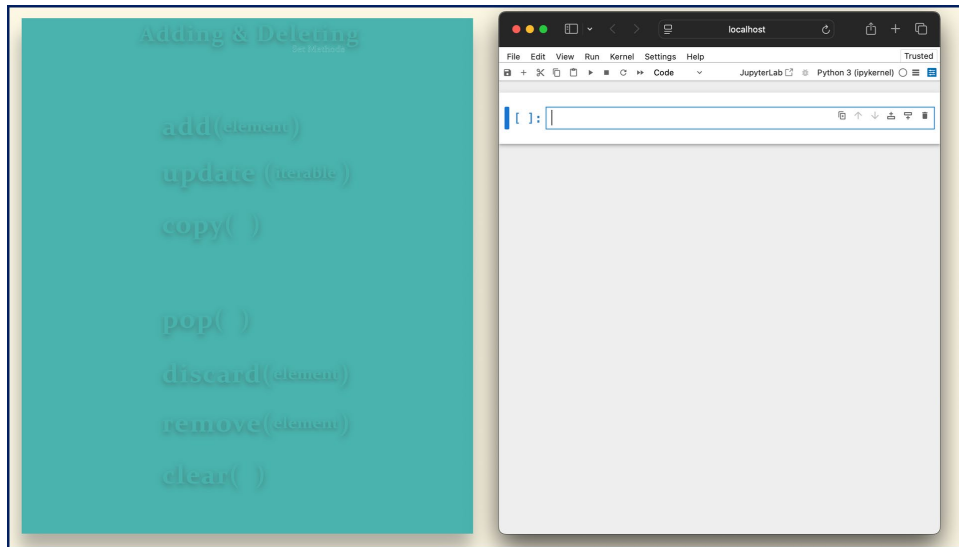
S1

1020304050

S2

1020304050

S2=S1.copy()



Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
{ }:
```

S1

10	20	30	40	50
----	----	----	----	----

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
{ }:
```

S1

10	20	30	40	50
----	----	----	----	----

S1.pop()

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.pop()
[2]: 50
[ ]:
```

S1

10	20	30	40
----	----	----	----

S1.pop()

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

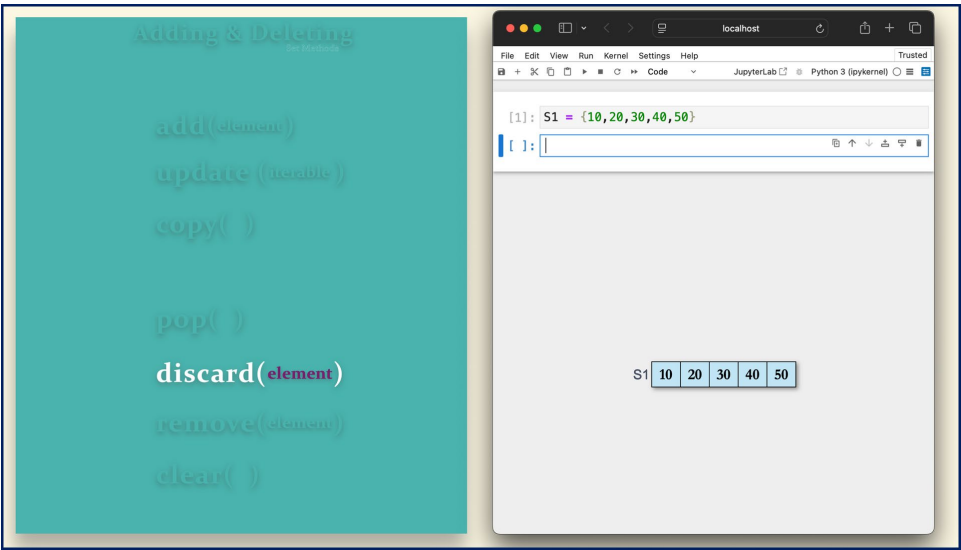
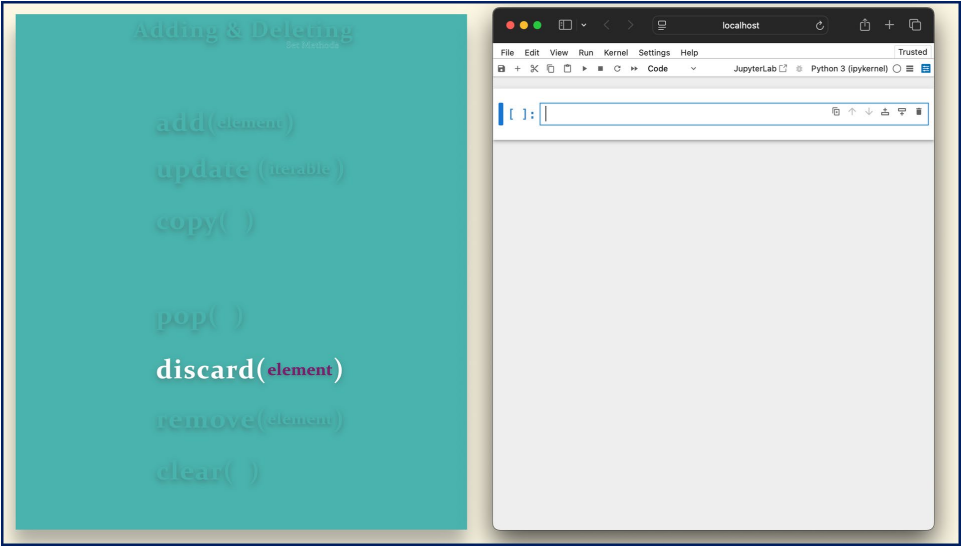
clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.pop()
[2]: 50
[3]: S1.pop()
[3]: 20
[ ]:
```

S1

10	30	40
----	----	----

S1.pop()



Adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

S1.discard()

Adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
{ }:
```

S1 10 20 30 40 50

S1.discard(30)

Adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[2]: S1.discard(30)
```

```
{ }:
```

S1 10 20 40 50

S1.discard(30)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help Trusted

Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.discard(30)

[3]: S1

[3]: {10, 20, 40, 50}

[]:

S1

10	20	40	50
----	----	----	----

S1.discard(30)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help Trusted

Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.discard(70)

[]:

S1

10	20	30	40	50
----	----	----	----	----

S1.discard(70)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.discard(70)
[3]: S1
[3]: {10, 20, 30, 40, 50}
```

S1

10	20	30	40	50
----	----	----	----	----

S1.discard(70)

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

S1

10	20	30	40	50
----	----	----	----	----

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

S1.remove(70)

Adding & Deleting

Set Methods

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.remove(70)

KeyError

most recent call last

Cell In[2], line 1

----> 1 S1.remove(70)

KeyError: 70

Traceback (

S1 10 20 30 40 50

S1.remove(70)

adding & Deleting

Set Methods

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}

[2]: S1.remove(30)

[]:

S1 10 20 40 50

S1.remove(30)

Adding & Deleting

`add(element)`

`update (element )`

`copy( )`

`pop( )`

`discard(element)`

`remove(element)`

`clear( )`

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: S1 = {10,20,30,40,50}
[2]: S1.remove(30)
[3]: S1
[3]: {10, 20, 40, 50}

In [ ]:

S1 10 20 40 50

S1.remove( 30 )
```

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1 10 20 30 40 50

S1.remove(20,30)

Adding & Deleting

add(element)

update (iterable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
```

```
[2]: S1.remove(20,30)
```

```
Traceback (most recent call last):
```

```
Cell In[2], line 1
```

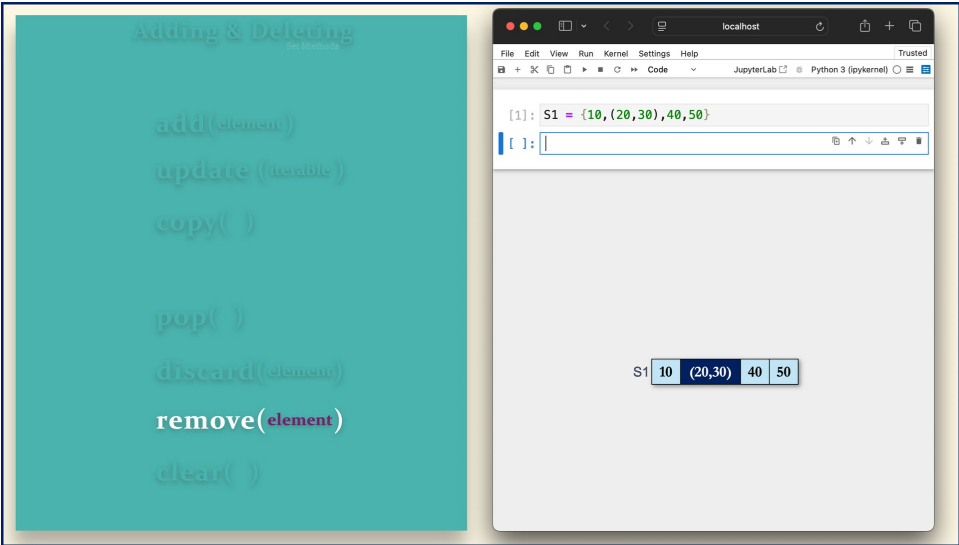
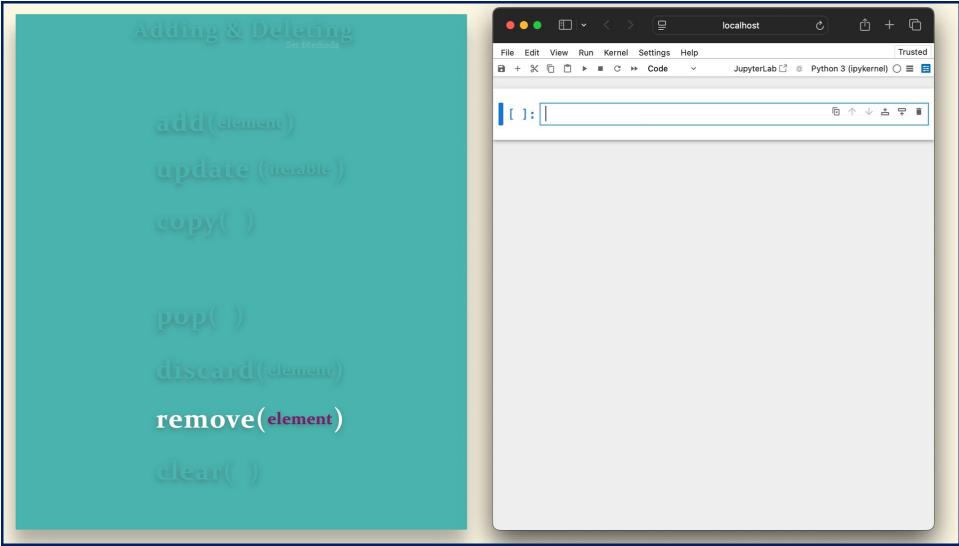
```
----> 1 S1.remove(20,30)
```

```
TypeError: set.remove() takes exactly one argument (2 given)
```

```
[ ]:
```

S1 10 20 30 40 50

S1.remove(20,30)



Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,(20,30),40,50}
```

```
{ }:
```

S1 10 (20,30) 40 50

S1.remove((20,30))

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,(20,30),40,50}
```

```
[2]: S1.remove((20,30))
```

```
{ }:
```

S1 10 40 50

S1.remove((20,30))

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10, (20,30), 40, 50}
[2]: S1.remove((20,30))
[3]: S1
[3]: {10, 40, 50}
```

S1

10	40	50
----	----	----

S1.remove((20,30))

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[ ]:
```

Adding & Deleting

Methods

- `add(element)`
- `update (iterable)`
- `copy( )`
- `pop( )`
- `discard(element)`
- `remove(element)`
- `clear( )`

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

adding & Deleting

Methods

- `add(element)`
- `update (iterable)`
- `copy( )`
- `pop( )`
- `discard(element)`
- `remove(element)`
- `clear( )`

```
[1]: S1 = {10,20,30,40,50}
```

```
[ ]:
```

S1

10	20	30	40	50
----	----	----	----	----

S1.clear()

Adding & Deleting

add(element)

update (element)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.clear()
[ ]:
```

S1 -

S1.clear()

Adding & Deleting

add(element)

update (mutable)

copy()

pop()

discard(element)

remove(element)

clear()

```
[1]: S1 = {10,20,30,40,50}
[2]: S1.clear()
[3]: S1
[3]: set()
[4]:
```

S1

S1.clear()